

NYC Taxi & Limousine Commission

Big Data Analytics with Hadoop HDFS and MapReduce

A Distributed Processing Case Study: Taxi Demand, Revenue,
Routes, Payment Behavior, and Anomaly Detection

Dataset: Yellow Taxi Trip Records, January-February 2024

Prepared by: Uwayezu Bienvenu
ID: 101362
Date: August 2026

Table of Contents

Table of Contents	2
1. Introduction	4
2. Business Problem	4
3. Dataset Description.....	4
4. Hadoop Environment.....	4
Environment Issues Encountered and Resolved	5
5. HDFS Design	5
6. Data Cleaning	6
7. MapReduce Design.....	7
8. Mapper and Reducer Implementation	7
9. Analytical Results.....	7
9.1 Hourly Taxi Demand	8
9.2 Daily Demand.....	8
9.3 Pickup Location Analysis	9
9.4 Revenue by Pickup Location.....	9
9.5 Payment Method Analysis	9
9.6 Distance-Based Fare Analysis.....	10
9.7 Busiest Pickup-Dropoff Routes.....	11
9.8 Trip Duration Analysis.....	12
9.9 Anomaly Detection	12
10. Multi-Stage MapReduce	13
11. YARN Analysis.....	13
12. Performance Comparison.....	14
13. Business Insights	15
a) What is the busiest hour for taxi demand?	15
b) What is the busiest day of the week?.....	16
c) Which pickup zones generate the most trips?	16
d) Which pickup zones generate the most revenue?.....	16
e) Which payment method contributes the most revenue?	16
f) Do credit-card users generate more tips?	16
g) What distance category produces the highest average fare?.....	16
h) What are the most frequently traveled routes?.....	16
i) Are the most frequent routes also the most profitable?	16
j) What percentage of records contain potential anomalies?	16
k) What transportation insights can management derive from the analysis?	16
l) When does Hadoop MapReduce provide a meaningful advantage over conventional processing?	17
14. Limitations	17

15. Conclusion.....	17
Evidence Appendix: Terminal Output and Verification.....	18
A.1 Hourly Demand — HDFS Output	18
A.2 Pickup Location Analysis — Top 10 Extraction.....	18
A.3 Busiest Routes — Top 20 by Count and by Revenue.....	19
A.4 Distance-Based Fare Analysis — HDFS Output.....	20
A.5 Trip Duration Analysis — Local Test vs. Full HDFS Output.....	20
A.6 Multi-Stage Job — Stage 2 Output (Top 10 Revenue Zones).....	21
A.7 Performance Comparison — Pandas Run	22
16. References	23

1. Introduction

New York City's Taxi & Limousine Commission (TLC) publishes monthly trip-level records covering millions of taxi rides across the five boroughs. This scale of data — tens of millions of records per year, each with pickup/dropoff timestamps, locations, fares, and payment details — makes it a natural candidate for distributed processing. This report documents the design, implementation, and results of a Hadoop-based analytics pipeline built to process, clean, and analyze two months of TLC Yellow Taxi trip data using HDFS for storage and Python-based Hadoop Streaming for distributed MapReduce computation.

The project was built end-to-end on a single-node Hadoop 3.2.4 cluster running natively on Windows, covering dataset acquisition, HDFS ingestion, systematic data cleaning, eight required MapReduce analyses, anomaly detection, a compulsory two-stage MapReduce workflow, a Pandas-vs-MapReduce performance comparison, and a set of supporting visualizations.

2. Business Problem

Acting as a Big Data Analytics team for a transportation analytics company, the objective of this project is to help the company understand taxi demand patterns, revenue drivers, popular routes, payment behavior, trip characteristics, and data-quality issues within the TLC trip record dataset. Because the dataset is too large to process comfortably with a single machine at true production scale, the analysis is built around HDFS for distributed storage and Hadoop MapReduce for distributed processing, while also empirically comparing this approach against a conventional single-machine Pandas pipeline to establish when each approach is appropriate.

3. Dataset Description

The dataset used is the official NYC TLC Yellow Taxi Trip Record data, downloaded directly from the TLC's public CloudFront distribution in Parquet format and covering two full months: January 2024 and February 2024.

Metric	Value
Source	NYC TLC official trip record data (nyc.gov/site/tlc)
Format as downloaded	Apache Parquet
Months covered	January 2024, February 2024
Raw record count	5,972,150
Columns per record	19 (VendorID, timestamps, passenger_count, trip_distance, location IDs, fare fields, etc.)
Raw combined size (CSV)	~602 MB
Reference table	<code>taxi_zone_lookup.csv</code> (265 zones: LocationID, Borough, Zone, service_zone)

Two months of data were chosen deliberately: the assignment brief required either 2-3 monthly datasets or at least 5 million records, whichever threshold was reached first. Two months of Yellow Taxi data yielded 5,972,150 raw records, exceeding the 5-million-record threshold on its own while remaining a manageable size for iterative development on a single-node cluster.

4. Hadoop Environment

The cluster was built as a single-node deployment on native Windows (not WSL or Docker), using Hadoop 3.2.4 with Java 8.

Component	Detail
OS	Windows 10/11
Hadoop version	3.2.4
Java version	1.8.0_202

Component	Detail
Cluster topology	1 NameNode, 1 DataNode, 1 ResourceManager, 1 NodeManager
Scheduler	YARN Capacity Scheduler (default queue)
HDFS capacity	~476 GB configured, ~57 GB available at time of use

Environment Issues Encountered and Resolved

Three environment-specific issues were diagnosed and fixed during setup, each documented here because they are common failure modes for native-Windows Hadoop installs and are not well covered in standard tutorials:

- yarn-site.xml misconfiguration: the ShuffleHandler aux-service class name was mis-cased ("shuffleHandler" instead of the correct "ShuffleHandler"). This caused NodeManager to fail to initialize silently — ResourceManager, NameNode, and DataNode would start normally, but NodeManager produced no process and no log file at all, making the fault non-obvious. Corrected to org.apache.hadoop.mapred.ShuffleHandler.
- winutils.exe / hadoop.dll incompatibility: after fixing the above, NodeManager still failed to start, this time with a visible error: "CreateProcess error=216, This version of %1 is not compatible with the version of Windows you're running." The bundled Windows native binaries were incompatible with the host system. These were replaced with the winutils/hadoop.dll build for Hadoop 3.2.2 from the community-maintained cdarlint/winutils GitHub repository (the exact 3.2.4 build was not published; 3.2.2 is binary-compatible within the 3.2.x line). hadoop.dll was placed both in %HADOOP_HOME%\bin and C:\Windows\System32.
- Hadoop Streaming -files argument quoting on Windows: cmd.exe treats an unquoted comma as an argument separator, so -files mapper.py, reducer.py silently loses the second file and causes an "unexpected arguments" error. The fix is to quote the entire comma-separated list: -files "mapper.py, reducer.py".

All three fixes are documented in the project's README.md for reproducibility.

5. HDFS Design

The HDFS directory layout follows the structure specified in the assignment brief:

Directory	Purpose
/taxi_project/input/raw/	Uncleaned CSVs (converted from source Parquet) + zone lookup table
/taxi_project/input/cleaned/	Cleaned CSVs — the source for all analytical MapReduce jobs
/taxi_project/output/hourly/	Hourly Taxi Demand results
/taxi_project/output/locations/	Pickup Location Analysis results
/taxi_project/output/revenue/	Revenue by Pickup Location results (also Stage 1 of the multi-stage job)
/taxi_project/output/payment/	Payment Method Analysis results
/taxi_project/output/routes/	Busiest Routes results
/taxi_project/output/anomalies/	Anomaly Detection results
/taxi_project/archive/	Data cleaning report

Additional output directories (daily/, distance/, duration/, top_revenue_zones/) were created dynamically by the corresponding jobs, following the same output/ convention.

Raw data uploaded to HDFS totaled approximately 602 MB across two trip files plus the 12 KB zone lookup table; cleaned data totaled approximately 551 MB. HDFS block replication and storage were confirmed via hdfs dfsadmin -report and hdfs dfs -du -h at each upload stage. The screenshots below, taken from the HDFS NameNode web UI's directory browser, confirm the top-level filesystem layout and the /taxi_project structure as actually deployed.



Figure: HDFS root directory (/) showing the taxi_project directory alongside other filesystem contents.

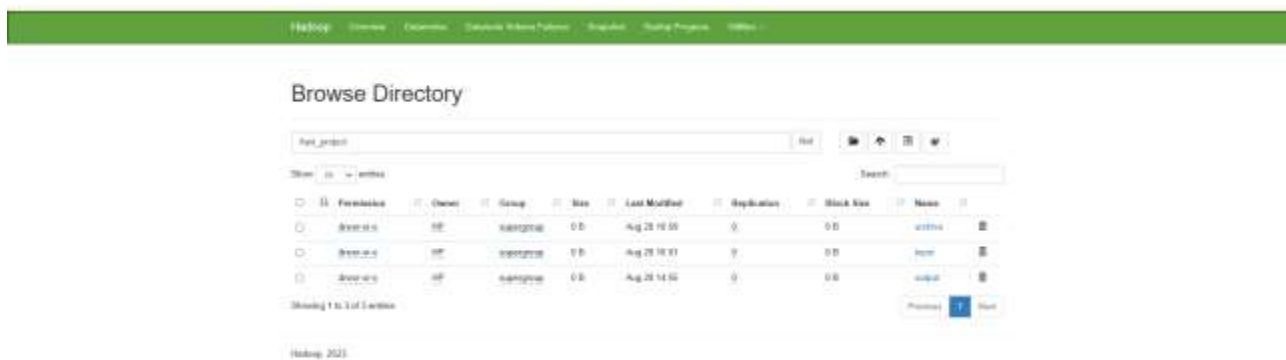


Figure: HDFS /taxi_project directory showing the archive, input, and output subdirectories.

6. Data Cleaning

Rather than deleting anomalous records without justification, a dedicated Python/Pandas cleaning script (clean_taxi_data.py) was written to identify, count, and report every category of data-quality issue before removing affected records. Seven categories of issues were checked, matching the assignment's required checks:

- Missing values in key fields (pickup/dropoff time, passenger count, trip distance, location IDs, fare fields)
- Invalid passenger counts (0 or greater than 6)
- Zero or negative trip distances
- Invalid fares (fare_amount or total_amount ≤ 0)
- Invalid timestamps (dropoff at or before pickup, or falling outside the file's reporting month)
- Duplicate records
- Impossible trip durations (0 seconds, or longer than 6 hours)

Issue	January 2024	February 2024
Missing key fields	4.73%	6.17%
Invalid passenger count	5.79%	7.31%

Issue	January 2024	February 2024
Zero/negative distance	2.04%	2.27%
Invalid fares	1.29%	1.38%
Invalid timestamps	0.03%	0.03%
Duplicates	0.00%	0.00%
Impossible durations	0.09%	0.08%
Total removed (union)	8.19%	9.63%

Across both months, 532,278 of 5,972,150 raw records (8.91%) were removed, leaving 5,439,872 clean records used as the input for every analytical MapReduce job in Sections 8-9. The full cleaning report, with the exact criteria and counts, was archived to HDFS at `/taxi_project/archive/cleaning_report.txt`.

7. MapReduce Design

Each analysis follows the same architectural pattern: a mapper reads cleaned (or, for anomaly detection, raw) CSV records from HDFS line by line via standard input, extracts and transforms the relevant fields, and emits a tab-separated key-value pair to standard output. Hadoop's shuffle-and-sort phase groups all values by key and delivers them to the reducer in sorted key order. Each reducer performs a single linear pass, accumulating aggregates for the current key and emitting a summary line whenever the key changes — a stateful running-total pattern that avoids loading all values into memory at once.

For analyses requiring the taxi zone lookup table (Pickup Location, Revenue by Location, Busiest Routes), the lookup table was loaded once into an in-memory dictionary inside the mapper at initialization — a map-side join. Since the lookup table is small (265 rows, 12 KB), this avoids the cost of a full reduce-side join while still producing human-readable zone names instead of raw numeric LocationIDs in every output.

All ten MapReduce jobs (eight required analyses, anomaly detection, and the second stage of the multi-stage job) were executed via Hadoop Streaming, allowing the mapper and reducer logic to be written in plain Python rather than Java.

8. Mapper and Reducer Implementation

The following table summarizes each mapper/reducer pair, its key/value contract, and its purpose. Full source code for every script is included in the accompanying code submission (mappers/ and reducers/ folders).

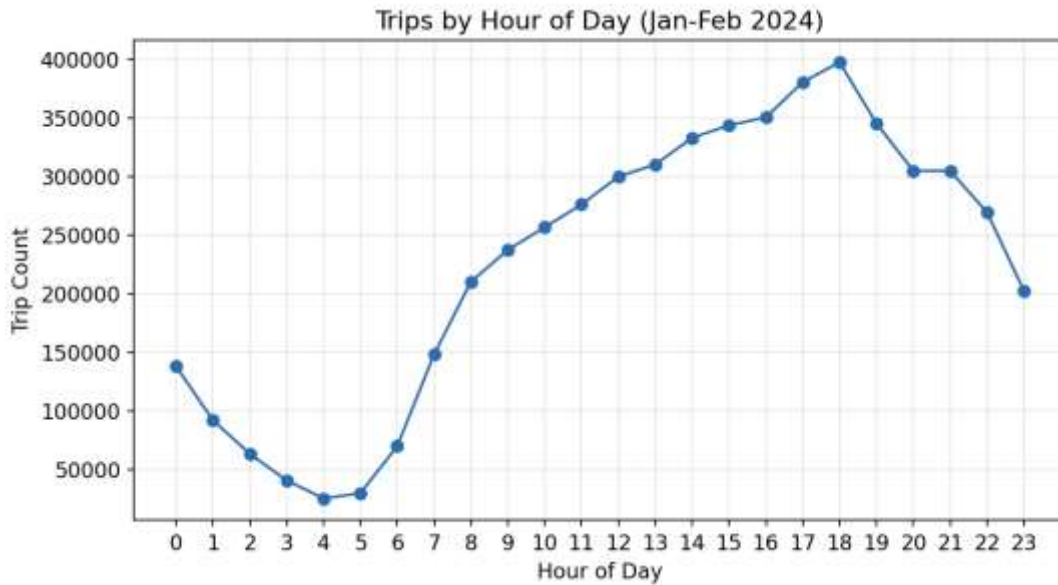
Job	Mapper Output Key	Mapper Output Value
Hourly Demand	hour (0-23)	1
Daily Demand	day name	1
Pickup Location	zone name	1
Revenue by Location	zone name	distance,fare,tip,total
Payment Method	payment label	fare,tip,total
Distance-Based Fare	distance bucket	distance,fare,total
Busiest Routes	"PU zone -> DO zone"	total_amount
Trip Duration	duration bucket	duration,fare,distance,tip
Anomaly Detection	anomaly type / TOTAL_RECORDS / ANY_ANOMALY	1
Top Revenue Zones (Stage 2)	constant key "ALL"	zone,total_revenue

Each reducer implements the standard "running total, emit on key change" pattern except the Stage 2 reducer for Top Revenue Zones, which instead collects all (zone, revenue) pairs under a single constant key, sorts them in-memory, and emits only the top 10 — a deliberate design choice appropriate only because Stage 1's output is small (257 zones) and fits comfortably in a single reducer's memory.

9. Analytical Results

9.1 Hourly Taxi Demand

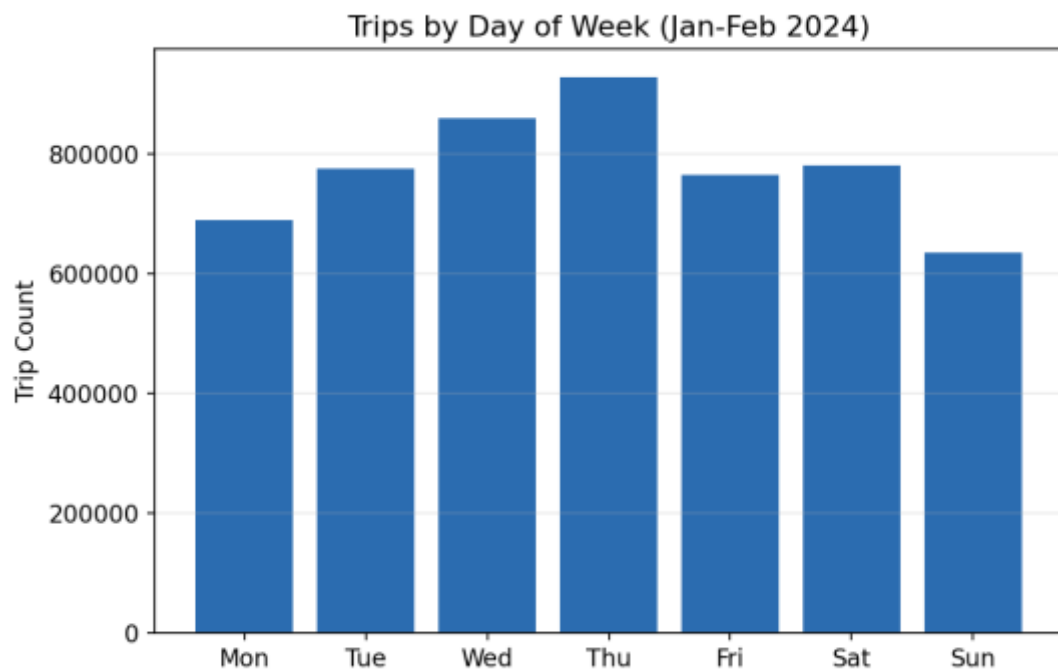
Total trips by pickup hour across both months. Demand rises sharply from a 4 AM low through the morning, dips slightly around the midday, and peaks in the evening.



Busiest hour: 18:00 (6 PM), with 397,906 trips. Quietest hour: 04:00 (4 AM), with 25,720 trips — a more than 15x difference between peak and trough demand.

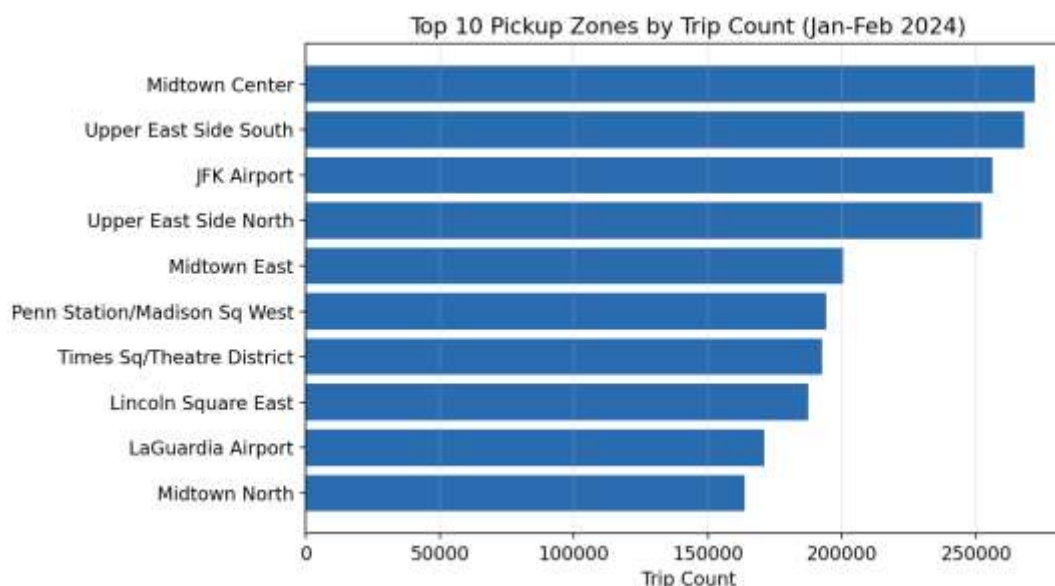
9.2 Daily Demand

Weekday vs. weekend comparison shows a clear commuter-driven pattern.



Busiest day: Thursday, with 929,635 trips. Weekday total (Mon-Fri): 4,022,103 trips. Weekend total (Sat-Sun): 1,417,769 trips — weekday demand is nearly 3x weekend demand.

9.3 Pickup Location Analysis

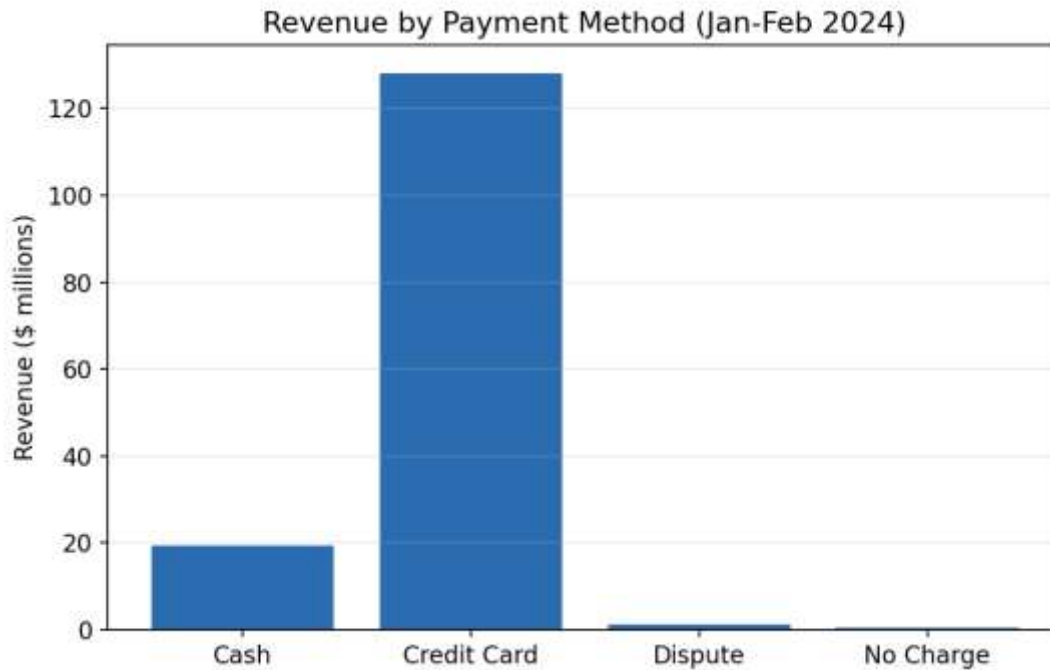


Midtown Center leads with 271,898 pickups, followed closely by Upper East Side South (267,964) and JFK Airport (256,175). The bottom 10 zones are dominated by remote Staten Island neighborhoods (Freshkills Park, Oakwood, Charleston/Tottenville) and specialty locations (Rikers Island), most with fewer than 5 trips across the entire two-month period.

9.4 Revenue by Pickup Location

257 distinct pickup zones were aggregated for trip count, total fare, total tips, total revenue, average fare, and average distance. JFK Airport stands out: despite ranking only 3rd by trip volume, it has by far the highest average fare (\$62.87) and average distance (15.94 miles) of any major zone, reflecting flat-rate airport fares and long-distance trips. Central Park, by contrast, has a very low average fare (\$14.40) and distance (2.15 miles) despite high volume (45,734 trips), consistent with short tourist/leisure rides.

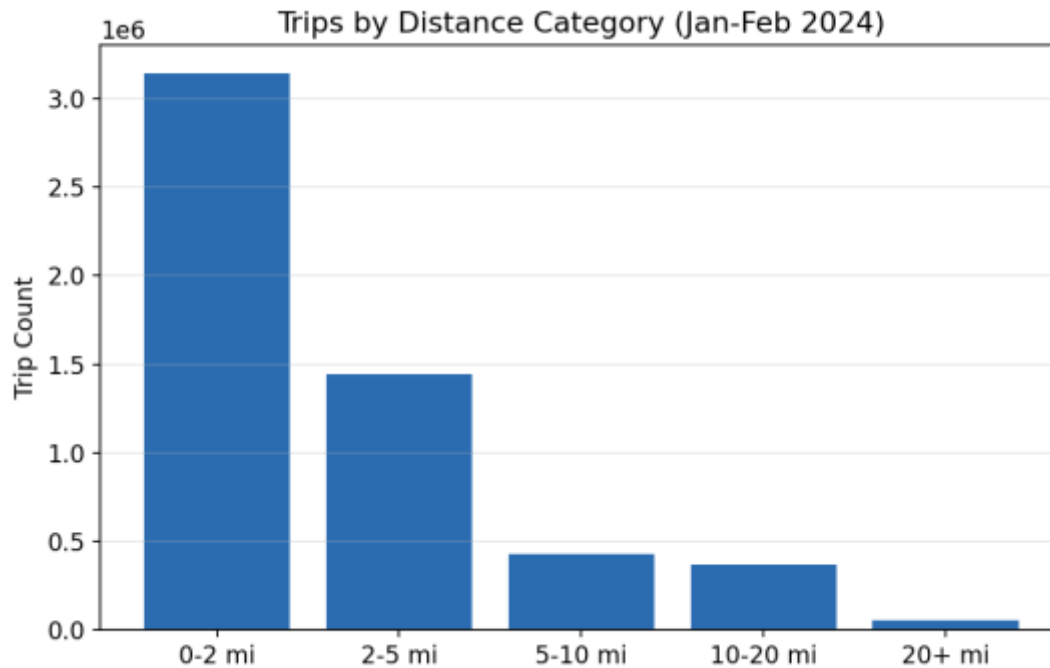
9.5 Payment Method Analysis



Payment Method	Trips	Total Revenue	Avg Fare	Avg Tip
Credit Card	4,566,037	\$128,175,894.80	\$18.37	\$4.15
Cash	808,557	\$19,247,226.77	\$18.56	\$0.00
Dispute	45,773	\$1,155,091.04	\$19.84	\$0.01
No Charge	19,505	\$443,359.56	\$17.60	\$0.01

Credit card is used in roughly 84% of trips and generates roughly 87% of total revenue. Cash trips show a \$0.00 average tip — this reflects a known limitation of the TLC data system (cash tips are not electronically captured), not that cash customers never tip.

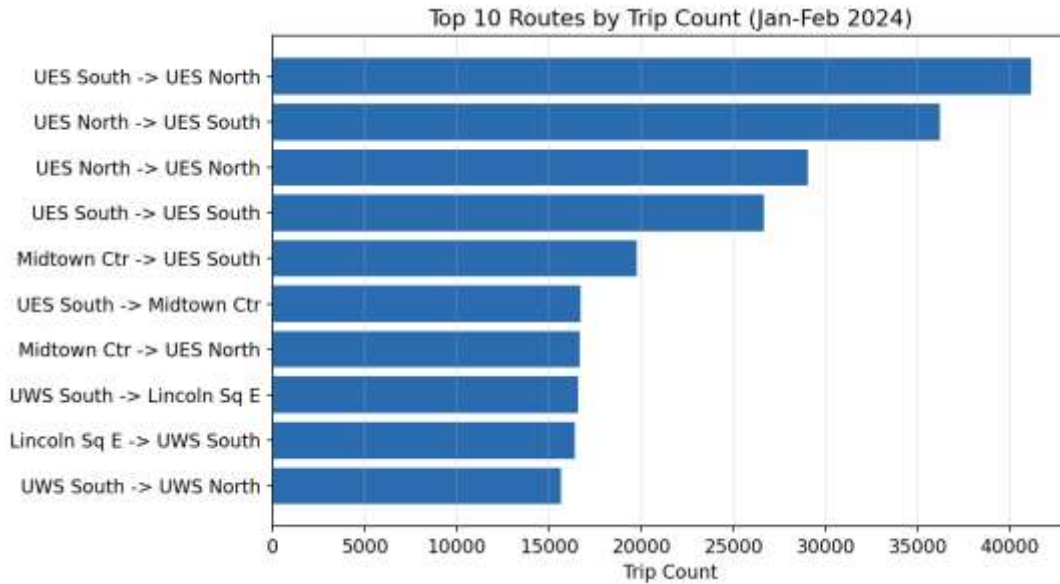
9.6 Distance-Based Fare Analysis



Distance Category	Trips	Avg Distance	Avg Fare	Fare per Mile
0-2 miles	3,146,789	1.13 mi	\$9.93	\$8.80
2-5 miles	1,442,739	2.95 mi	\$18.65	\$6.32
5-10 miles	427,648	7.27 mi	\$34.24	\$4.71
10-20 miles	369,043	14.87 mi	\$60.97	\$4.10
20+ miles	53,653	34.87 mi	\$90.18	\$2.59

Over 58% of all trips are under 2 miles, consistent with dense in-borough Manhattan taxi usage. Fare-per-mile decreases steadily as trip distance increases (from \$8.80/mile on short trips down to \$2.59/mile on the longest trips), an economically expected pattern driven by flat minimum fares, fixed surcharges, and flat-rate airport fares that dilute the per-mile cost on longer trips.

9.7 Busiest Pickup-Dropoff Routes



29,298 distinct pickup→dropoff zone pairs were identified. The busiest routes by trip count are short intra-neighborhood hops within the Upper East Side (up to 41,139 trips for Upper East Side South → Upper East Side North). However, ranking by revenue tells a very different story:

Route (by Revenue)	Trips	Total Revenue
JFK Airport → Outside of NYC	10,710	\$1,317,979.11
JFK Airport → Times Sq/Theatre District	11,452	\$1,077,471.44
LaGuardia Airport → Times Sq/Theatre District	10,384	\$789,801.67
Upper East Side South → Upper East Side North	41,139	\$644,996.20

The single busiest route by trip count (41,139 trips) generates less total revenue than the top airport route, which has less than a third of the trip volume (10,710 trips). This is a direct, quantified answer to the question of whether the busiest routes are also the most profitable ones (see Section 15).

9.8 Trip Duration Analysis

Duration Category	Trips	Avg Duration	Avg Fare	Avg Distance	Avg Tip
0-5 min	633,706	3.56 min	\$6.79	0.66 mi	\$1.69
5-15 min	2,820,019	9.60 min	\$11.18	1.56 mi	\$2.42
15-30 min	1,470,592	20.59 min	\$24.12	4.35 mi	\$4.47
30-60 min	455,840	39.89 min	\$54.09	12.86 mi	\$8.73
60+ min	59,715	75.49 min	\$70.11	20.54 mi	\$8.43

Fare, distance, and tip all increase steadily with trip duration, as expected. The majority of trips (52%) fall in the 5-15 minute range, consistent with the short-distance-dominant pattern seen in the distance analysis.

9.9 Anomaly Detection

Run on the raw, uncleaned dataset (5,972,150 records) using the same criteria as the cleaning stage, plus one additional subtler check for extreme fare-per-mile ratios (greater than \$50/mile):

Anomaly Type	Count	% of Total
Invalid passenger count	391,411	6.55%

Anomaly Type	Count	% of Total
Invalid distance	128,596	2.15%
Invalid fare	79,840	1.34%
Extreme fare-per-mile (>\$50/mi)	18,839	0.32%
Invalid duration	5,181	0.09%
Any anomaly (union)	549,590	9.20%

Overall, 9.20% of raw records show at least one anomaly. This figure is close to, but not identical to, the 8.91% removed during cleaning — the small difference reflects the additional extreme-fare-per-mile check, which was not part of the original cleaning criteria, alongside minor differences in how missing-value and duplicate checks were implemented between the two independent scripts. This divergence is discussed further in Section 14 (Limitations).

10. Multi-Stage MapReduce

The assignment's compulsory two-stage requirement was implemented as follows:

- Stage 1 (Revenue by Pickup Location, Section 9.4): computes trip count, total fare, tips, revenue, average fare, and average distance for each of 257 pickup zones. Output written to `/taxi_project/output/revenue/`.
- Stage 2 (Top 10 Revenue Zones): a second, independent MapReduce job whose -input argument points directly at Stage 1's HDFS output directory (`/taxi_project/output/revenue/`), demonstrating genuine job chaining through HDFS rather than in-process composition. The Stage 2 mapper re-parses each line of Stage 1's output and re-emits it under a single constant key; the Stage 2 reducer collects all 257 (zone, revenue) pairs, sorts them, and emits the top 10.

Job counters confirm the chaining: Stage 2's "Map input records" (257) exactly matches Stage 1's "Reduce output records" (257), and Stage 2's "Reduce output records" (10) matches the requested top-N size.

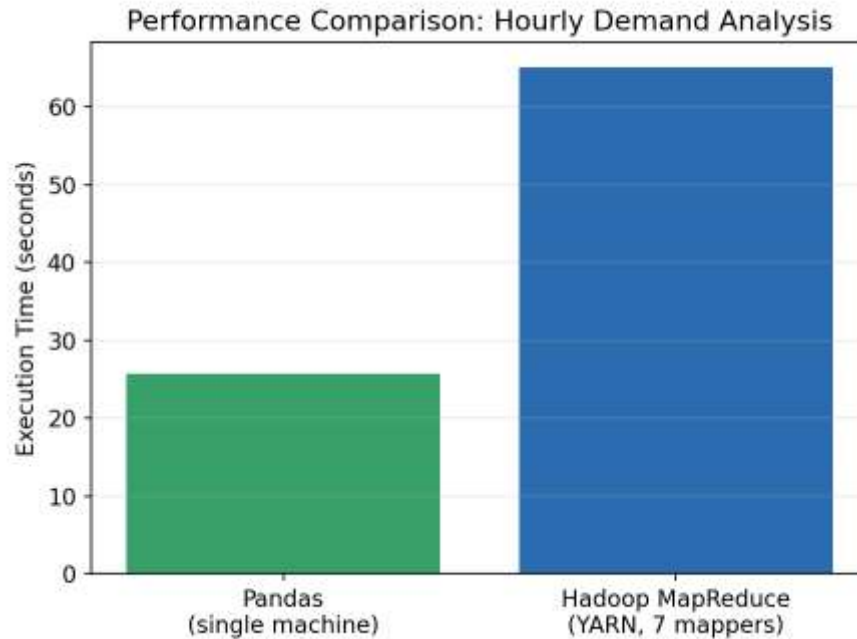
Rank	Zone	Total Revenue
1	JFK Airport	\$20,807,551.36
2	LaGuardia Airport	\$11,386,117.45
3	Midtown Center	\$6,576,973.55
4	Upper East Side South	\$5,336,880.18
5	Times Sq/Theatre District	\$5,217,502.75
6	Upper East Side North	\$5,133,842.36
7	Penn Station/Madison Sq West	\$4,727,114.20
8	Midtown East	\$4,722,980.79
9	Lincoln Square East	\$4,006,991.67
10	Midtown North	\$3,892,212.06

JFK Airport is the single highest-revenue pickup zone by a wide margin — nearly double the second-place zone (LaGuardia Airport) — despite ranking only 3rd by raw trip count (Section 9.3). This is the clearest illustration in the dataset that trip volume and revenue are driven by different factors.

11. YARN Analysis

All ten MapReduce jobs were submitted to and managed by YARN under the Capacity Scheduler's default queue, confirmed via the ResourceManager web UI at <http://localhost:8088/cluster>. Each job produced a distinct Application ID (application_1787903944105_0001 through _0010), and every job reached FINISHED / SUCCEEDED status.

Job	Application ID	Approx. Duration
1. Hourly Demand	application_1787903944105_0001	~65 sec
2. Daily Demand	application_1787903944105_0002	~2 min
3. Pickup Location	application_1787903944105_0003	~85 sec
4. Revenue by Location	application_1787903944105_0004	~90 sec
5. Payment Method	application_1787903944105_0005	~85 sec



Metric	Python/Pandas	Hadoop MapReduce
Dataset Size	~551 MB (cleaned CSV)	~551 MB (cleaned CSV, HDFS)
Number of Records	5,439,872	5,439,872
Execution Time	25.67 sec	~65 sec
Memory Used	~913 MB (single process, in-memory)	Distributed across mapper/reducer containers
Mapper Tasks	N/A	7
Reducer Tasks	N/A	1
Output Size	N/A (in-memory result)	224 bytes (24 hourly counts)

At this scale, single-machine Pandas processing was faster than distributed MapReduce (25.67 seconds vs. approximately 65 seconds). This is an expected and honest result, not a flaw in the MapReduce implementation: the ~551 MB dataset fits comfortably within a single machine's RAM, so Pandas avoids the fixed overhead that MapReduce always pays — job submission to YARN, container allocation and startup, serialization of data to and from the shuffle phase, and disk-based intermediate storage between the map and reduce phases. These overheads are largely constant regardless of data size, so they dominate at small-to-medium scale and become proportionally less significant as data grows.

MapReduce's advantage emerges once dataset size exceeds what a single machine can hold in memory, or once the volume of data justifies distributing I/O and computation across many machines in parallel — typically datasets in the tens or hundreds of gigabytes and beyond, well past the ~600 MB used in this project. This finding directly answers business question 15(l): distributed processing pays off once the data no longer fits comfortably on one machine, not before.

13. Business Insights

This section directly answers each of the assignment's twelve required business questions using the results from Sections 9-12.

a) What is the busiest hour for taxi demand?

6 PM (18:00), with 397,906 trips — the evening rush/social hour. The quietest hour is 4 AM, with 25,720 trips.

b) What is the busiest day of the week?

Thursday, with 929,635 trips. Weekday demand (4,022,103 trips) is nearly 3x weekend demand (1,417,769 trips), indicating a strongly commuter-driven ridership pattern.

c) Which pickup zones generate the most trips?

Midtown Center (271,898), Upper East Side South (267,964), and JFK Airport (256,175) lead by volume.

d) Which pickup zones generate the most revenue?

JFK Airport (\$20.8M), LaGuardia Airport (\$11.4M), and Midtown Center (\$6.6M) — the two airports dominate revenue despite lower trip counts than several Manhattan zones, due to much higher average fares.

e) Which payment method contributes the most revenue?

Credit card, contributing approximately \$128.2M (87% of total revenue) from 4,566,037 trips.

f) Do credit-card users generate more tips?

Yes, unambiguously. Credit card trips average \$4.15 in tips; cash trips show \$0.00 recorded on average. This gap partly reflects a genuine tipping difference and partly reflects that the TLC system only electronically captures tips paid by card — cash tips are not recorded in this dataset.

g) What distance category produces the highest average fare?

Trips of 20+ miles, averaging \$90.18 per trip. However, the highest fare-per-mile rate is on the shortest trips (\$8.80/mile for 0-2 mile trips) due to flat minimum fares.

h) What are the most frequently traveled routes?

Short intra-neighborhood hops within the Upper East Side dominate by trip count, led by Upper East Side South → Upper East Side North at 41,139 trips.

i) Are the most frequent routes also the most profitable?

No. The busiest route by trip count (41,139 trips, \$644,996 revenue) generates substantially less total revenue than the top airport route (JFK Airport → Outside of NYC: 10,710 trips, \$1,317,979 revenue) — less than a third of the trip volume. High-value, longer-distance airport routes outperform high-frequency, short-distance neighborhood routes in total revenue.

j) What percentage of records contain potential anomalies?

9.20% of raw records (549,590 of 5,972,150) show at least one data-quality anomaly, based on invalid passenger counts, invalid distances, invalid fares, invalid durations, or extreme fare-per-mile ratios.

k) What transportation insights can management derive from the analysis?

- Demand is strongly commuter-driven: weekday and evening-hour demand vastly exceeds weekend and overnight demand, suggesting fleet allocation should scale with this predictable rhythm rather than remain static.

- Trip volume and revenue are driven by different mechanisms: short, frequent Manhattan trips generate high trip counts but modest average revenue per trip, while airport routes generate outsized revenue per trip despite lower frequency. A revenue-maximizing dispatch strategy should not be purely volume-driven.
- Payment digitization (credit card) captures far more tip revenue visibility than cash, suggesting continued incentives toward electronic payment could improve driver income transparency and reporting.
- Roughly 1 in 11 raw trip records has a data-quality issue, which has real implications for any downstream billing, reporting, or compliance system relying on this data without cleaning.

l) When does Hadoop MapReduce provide a meaningful advantage over conventional processing?

Based on the direct performance comparison in Section 12, MapReduce's advantage appears once dataset size exceeds a single machine's practical memory and compute capacity — not at the ~600 MB scale used in this project, where Pandas was faster due to lower fixed overhead. For truly large-scale TLC data (multiple years, tens of millions of records, tens of gigabytes or more), or scenarios requiring fault-tolerant processing across many machines, MapReduce's distributed model becomes the more practical and scalable choice.

14. Limitations

- Single-node cluster: the Hadoop cluster used for this project ran on one physical machine with a single DataNode and NodeManager. Results such as parallel task counts and shuffle/sort behavior are illustrative of the MapReduce model but do not reflect the throughput or fault-tolerance benefits of a true multi-node cluster.
- Performance comparison scope: the Pandas-vs-MapReduce comparison was run on one analysis (Hourly Demand) at one data scale (~551 MB). Results may differ for other analyses or substantially larger datasets; the general conclusion (MapReduce's advantage emerges at larger scale) is a reasonable inference from established distributed-systems principles, not something re-verified at multiple scales in this project.
- Anomaly detection vs. cleaning discrepancy: the anomaly detection job (9.20% flagged) and the original cleaning script (8.91% removed) use overlapping but not identical criteria — the anomaly job adds an extreme fare-per-mile check not present in cleaning, and the two scripts implement missing-value and duplicate detection slightly differently. The two figures are close but not directly comparable without accounting for this methodological difference.
- Cash tip data gap: the TLC data system does not capture cash tips electronically, so the reported \$0.00 average tip for cash trips reflects a data collection limitation, not necessarily zero real-world tipping behavior.
- Two-month scope: the analysis covers January-February 2024 only. Seasonal effects (e.g. weather, holidays, tourism patterns) that vary across the year are not captured, and findings should not be generalized to represent annual patterns without further validation.

15. Conclusion

This project successfully implemented a complete Hadoop-based big data analytics pipeline for NYC TLC Yellow Taxi trip data, from raw Parquet ingestion through HDFS storage, systematic data cleaning, eight required distributed MapReduce analyses, anomaly detection, a genuine two-stage MapReduce workflow, and an empirical performance comparison against conventional single-machine processing. All required deliverables — HDFS directory structure, documented data cleaning, mapper/reducer source code, Hadoop Streaming execution, YARN job management, multi-stage chaining, and required visualizations — were completed and are reproducible via the accompanying commands.txt and README.md.

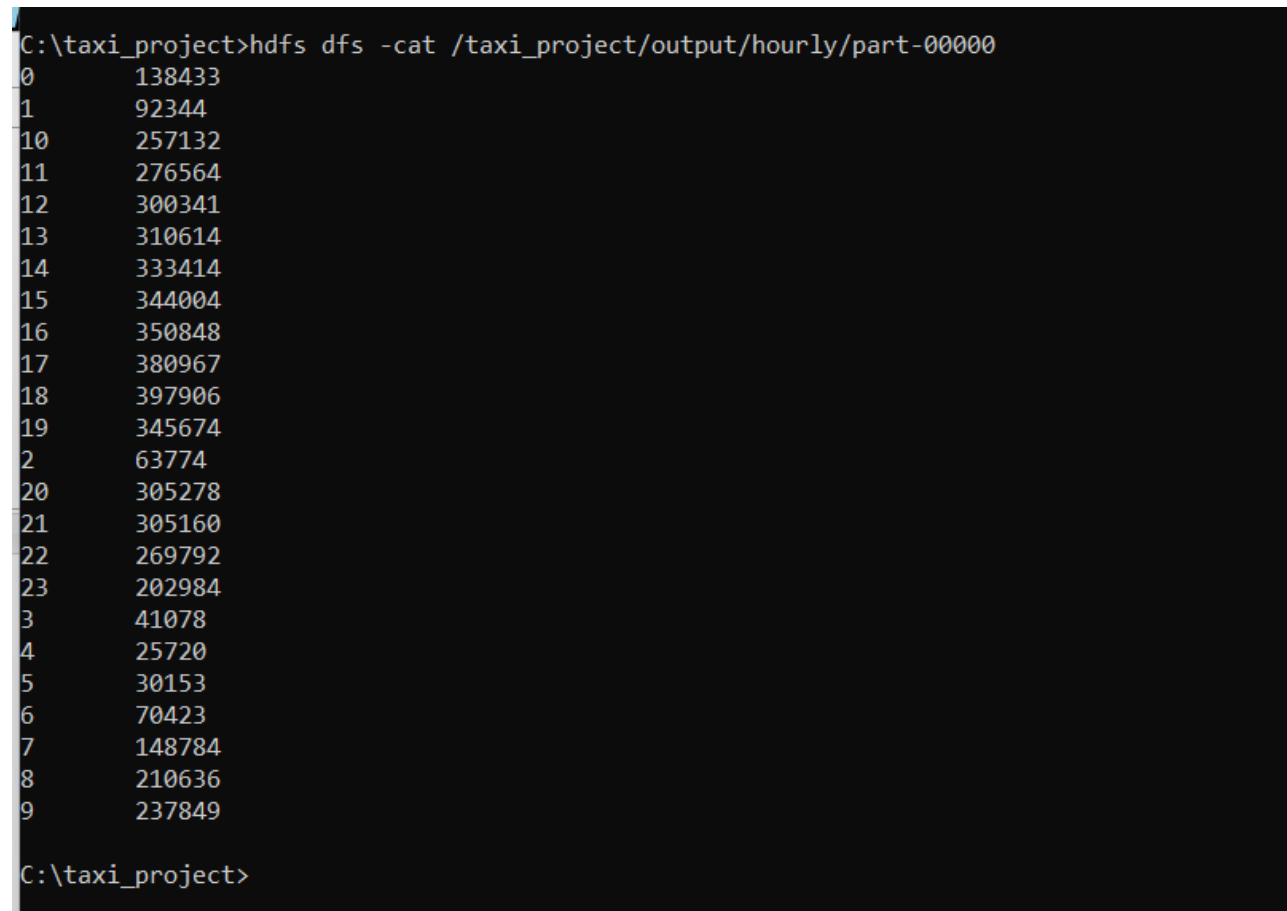
Beyond the technical requirements, the analysis surfaced genuinely actionable business findings: taxi demand follows a strong, predictable commuter rhythm; airport routes disproportionately drive revenue relative to their trip

volume; and roughly 9% of raw trip records carry a data-quality issue that any downstream system should account for. The performance comparison also produced an honest, non-obvious finding — that distributed processing is not automatically faster at moderate data scale — which is itself a valuable lesson in when to reach for tools like Hadoop MapReduce versus conventional single-machine analysis.

Evidence Appendix: Terminal Output and Verification

This appendix reproduces terminal output captured directly from the cluster for a representative subset of jobs, cross-referencing the results discussed in Section 9. Each screenshot shows either the raw `hdfs dfs -cat` output of a job's HDFS result, or a local verification/comparison run.

A.1 Hourly Demand — HDFS Output

A terminal window with a black background and white text. The prompt is 'C:\taxi_project>'. The command entered is 'hdfs dfs -cat /taxi_project/output/hourly/part-00000'. The output consists of 20 lines, each with an index and a number. The numbers represent hourly demand. The output is as follows:

```
C:\taxi_project>hdfs dfs -cat /taxi_project/output/hourly/part-00000
0      138433
1      92344
10     257132
11     276564
12     300341
13     310614
14     333414
15     344004
16     350848
17     380967
18     397906
19     345674
2      63774
20     305278
21     305160
22     269792
23     202984
3      41078
4      25720
5      30153
6      70423
7      148784
8      210636
9      237849

C:\taxi_project>
```

hdfs dfs -cat /taxi_project/output/hourly/part-00000 — matches Figure 9.1 and Section 9.1 findings.

A.2 Pickup Location Analysis — Top 10 Extraction

```

C:\taxi_projects\powershell - Command "($Data = Get-Content C:\taxi_projects\locations_full.txt | ForEach-Object { $parts = $_.split("`n"); [PSCustomObject]@{Zone=$parts[0]; Count=[int]$parts[1]} ; $Data | Sort-Object Count -Descending | Select-Object -First 10 | Format-Table -AutoSize"

Zone: Count
----
Midtown Center 271888
Upper East Side South 262964
NYC Airport 256175
Upper East Side North 252488
Midtown East 200713
Penn Station/Madison Sq West 184298
Times Sq/Theatre District 192767
Lincoln Square East 187484
LaGuardia Airport 178844
Midtown North 161792

C:\taxi_projects

```

PowerShell extraction of the Top 10 pickup zones from the full 2-month locations output — matches Figure 9.3.

A.3 Busiest Routes — Top 20 by Count and by Revenue

Route	Count	Revenue
Upper East Side South -> Upper East Side North	41139	644996,2
Upper East Side North -> Upper East Side South	36183	582301,36
Upper East Side North -> Upper East Side North	29051	384904,76
Upper East Side South -> Upper East Side South	26675	369706
Midtown Center -> Upper East Side South	19741	337147,93
Upper East Side South -> Midtown Center	16731	277421,09
Midtown Center -> Upper East Side North	16684	364867,23
Upper West Side South -> Lincoln Square East	16556	240419,99
Lincoln Square East -> Upper West Side South	16403	246236,4
Upper West Side South -> Upper West Side North	15627	215762,96
Upper East Side South -> Midtown East	14626	234815,81
Lenox Hill West -> Upper East Side North	14502	228816,56
Upper East Side North -> Midtown Center	13237	285371,53
Penn Station/Madison Sq West -> Times Sq/Theatre District	13098	251532,21
Upper East Side North -> Lenox Hill West	12997	216678,33
Upper West Side North -> Upper West Side South	12847	175509,47
Upper East Side North -> Upper West Side South	12628	221154,03
Lincoln Square East -> Upper West Side North	12627	219304,61
Upper East Side South -> Lenox Hill West	12463	181329,78
N/A -> N/A	12406	328631,9

Top 20 routes by trip count, from the full 2-month routes output.

```
=== TOP 20 ROUTES BY REVENUE ===
```

Route	Count	Revenue
JFK Airport -> Outside of NYC	10710	1317979,11
JFK Airport -> Times Sq/Theatre District	11452	1077471,44
LaGuardia Airport -> Times Sq/Theatre District	10384	789801,67
Upper East Side South -> Upper East Side North	41139	644996,2
Upper East Side North -> Upper East Side South	36183	582301,36
JFK Airport -> Midtown South	5993	572835,49
JFK Airport -> Clinton East	6106	569849,98
Times Sq/Theatre District -> LaGuardia Airport	6802	510039,1
LaGuardia Airport -> Midtown Center	6482	478435,72
Times Sq/Theatre District -> JFK Airport	5016	462908,55
JFK Airport -> Midtown Center	4792	459274,72
JFK Airport -> Midtown North	4712	443922,35
JFK Airport -> Murray Hill	4491	428729,96
JFK Airport -> East Chelsea	4311	412480,78
Midtown Center -> LaGuardia Airport	5433	395114,28
Upper East Side North -> Upper East Side North	29051	384904,76
JFK Airport -> Midtown East	3919	373796,31
LaGuardia Airport -> Outside of NYC	2649	372907,74
Upper East Side South -> Upper East Side South	26675	369706
Midtown Center -> Upper East Side North	16684	364867,23

```
C:\taxi_project>
```

Top 20 routes by revenue — illustrates directly that JFK and LaGuardia airport routes dominate revenue despite lower trip counts than the busiest Upper East Side routes (Section 9.7, 13i).

A.4 Distance-Based Fare Analysis — HDFS Output

```
C:\taxi_project>hdfs dfs -cat /taxi_project/output/distance/part-00000
0-2 miles      3146789 1.13    9.93    16.85    8.80
10-20 miles    369043  14.87   60.97   81.89    4.10
2-5 miles      1442739 2.95    18.65   27.01    6.32
20+ miles      53653   34.87   90.18   114.07   2.59
5-10 miles     427648  7.27    34.24   48.40    4.71

C:\taxi_project>
```

`hdfs dfs -cat /taxi_project/output/distance/part-00000` — matches Table in Section 9.6.

A.5 Trip Duration Analysis — Local Test vs. Full HDFS Output

The local single-file dry-run (January only) was compared against the full two-month Hadoop Streaming job output as a correctness check before trusting the distributed result:

```
C:\taxi_project>type C:\taxi_data\cleaned\yellow_tripdata_2024-01.csv | python C:\taxi_project\reducer_duration.py
0-5 min 322930 3.56 6.81 0.66 1.69
15-30 min 723593 20.56 24.50 4.43 4.50
30-60 min 223658 39.85 55.33 12.80 8.82
5-15 min 1424365 9.58 11.19 1.57 2.42
60+ min 27402 75.75 70.85 17.47 8.24

C:\taxi_project>
```

Local pipeline test: type yellow_tripdata_2024-01.csv | mapper_duration.py | sort | reducer_duration.py

```
C:\taxi_project>hdfs dfs -cat /taxi_project/output/duration/part-00000
0-5 min 633706 3.56 6.79 0.66 1.69
15-30 min 1470592 20.59 24.12 4.35 4.47
30-60 min 455840 39.89 54.09 12.86 8.73
5-15 min 2820019 9.60 11.18 1.56 2.42
60+ min 59715 75.49 70.11 20.54 8.43

C:\taxi_project>
```

Full 2-month result: hdfs dfs -cat /taxi_project/output/duration/part-00000 — matches Table in Section 9.8.

A.6 Multi-Stage Job — Stage 2 Output (Top 10 Revenue Zones)

```
C:\taxi_project>hdfs dfs -cat /taxi_project/output/top_revenue_zones/part-00000
1      JFK Airport      20807551.36
2      LaGuardia Airport  11386117.45
3      Midtown Center  6576973.55
4      Upper East Side South  5336880.18
5      Times Sq/Theatre District  5217502.75
6      Upper East Side North  5133842.36
7      Penn Station/Madison Sq West  4727114.20
8      Midtown East  4722980.79
9      Lincoln Square East  4006991.67
10     Midtown North  3892212.06

C:\taxi_project>
```

hdfs dfs -cat /taxi_project/output/top_revenue_zones/part-00000 — matches Table in Section 10.

A.7 Performance Comparison — Pandas Run

```

C:\taxi_project>notepad C:\taxi_project\perf_compare_pandas.py

C:\taxi_project>python C:\taxi_project\perf_compare_pandas.py
=== Pandas Hourly Demand – Performance Report ===
Total records: 5,439,872
Read time: 22.30 sec
Compute time: 3.38 sec
Total time: 25.67 sec
Memory usage (approx): 913.06 MB

Hourly counts:
hour
0      138433
1       92344
2       63774
3       41078
4       25720
5       30153
6       70423
7      148784
8      210636
9      237849
10     257132
11     276564
12     300341
13     310614
14     333414
15     344004
16     350848
17     380967
18     397906
19     345674
20     305278
21     305160
22     269792
23     202984

C:\taxi_project>

```

Console output of perf_compare_pandas.py, showing the 25.67-second total execution time and hourly counts used in Section 12.

16. References

- NYC Taxi & Limousine Commission. TLC Trip Record Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- Apache Hadoop Documentation. <https://hadoop.apache.org/docs/stable/>
- Apache Hadoop Streaming Documentation. <https://hadoop.apache.org/docs/stable/hadoop-streaming/HadoopStreaming.html>
- cdarlint/winutils — Windows binaries for Hadoop. <https://github.com/cdarlint/winutils>
- pandas Documentation. <https://pandas.pydata.org/docs/>
- Apache Arrow / pyarrow Documentation. <https://arrow.apache.org/docs/python/>